CARIOU Virginie BTS SIO SISR – 2ème année

Haute disponibilité d'une infrastructure web : répartition de charge, redondance du proxy et routage

Sommaire :

Objectifs

Schéma du réseau

Configuration VMnet8 : 192.168.100.0

Configuration de VMnet2 : 192.168.200.0

Installation de apache2 et openssh-server sur debian_web1 et debian_web2

Installation d'HAproxy et iptables sur debian_ha

Ajouter la 2ème carte VMnet2 à debian_ha

Changer la carte réseau de web1 et web2 sur VMnet2

Passer en IP fixe sur debian_ha 192.168.100.10, passerelle 192.168.100.2

Attribution d'une ip fixe sur debian_web1 et debian_web2

Personnalisation de la page d'accueil de debian_web1 et debian_web2

Configuration de HAProxy sur debian_ha

Tests : Simulation de panne sur un serveur web

Tests : SSH et Putty

Création d'une VM debian_ha2

Ajout d'une ip fixe pour debian_ha2 : 192.168.100.12, passerelle 192.168.100.2

Autoriser l'écoute sur une IP virtuelle

Copies des configurations de debian_ha sur debian_ha2

Autoriser l'IP virtuelle et installer Keepalived sur debian_ha

Changer les IP réelles de debian_ha 192.168.100.11 et 192.168.200.252

Configurer Keepalived sur debian_ha2 (BACKUP)

Tests de pannes sur HAproxy

Équilibrage des charges

Protection contre les attaques DDOS

Objectifs :

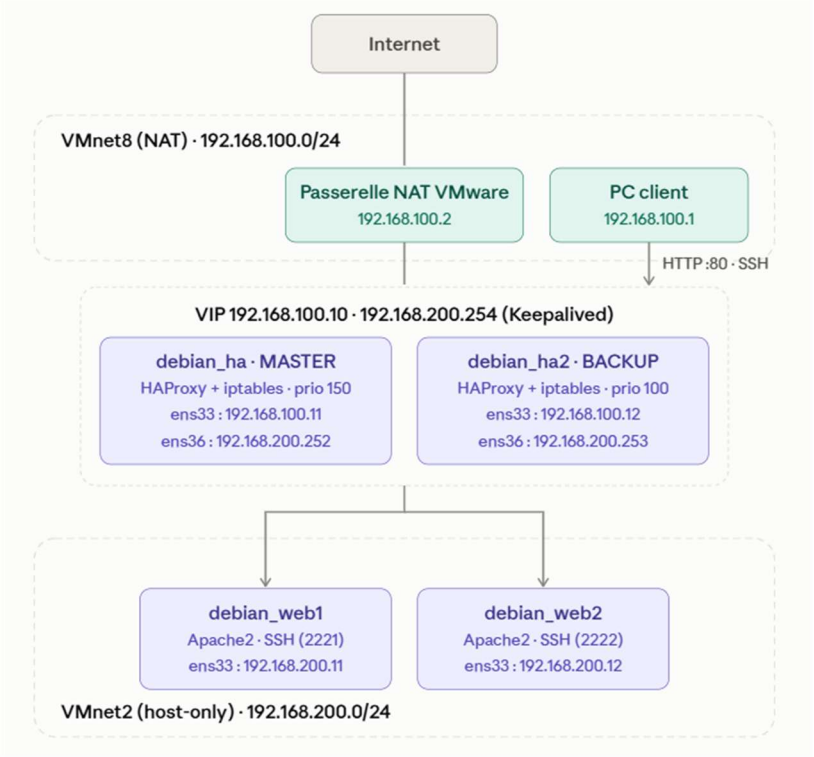
Mettre en place deux serveurs HAProxy (debian_ha et debian_ha2) chargés de répartir la charge entre deux serveurs web Apache (debian_web1 et debian_web2) et d'assurer la tolérance de panne : si un serveur web tombe, le trafic est automatiquement redirigé vers l'autre. La répartition est pondérée est testée : debian_web1 reçoit 30 % des requêtes et debian_web2 70 %. L'état des serveurs est supervisé via la page de statistiques de HAProxy.

Les serveurs HAProxy font également office de routeur : grâce au NAT (iptables), les serveurs web, situés sur un réseau interne, peuvent accéder à Internet. Une redirection de ports permet à un poste client de se connecter en SSH à debian_web1 (port 2221) et debian_web2 (port 2222) en passant par l'adresse du proxy.

Pour supprimer le point unique de défaillance que représente le proxy, les deux serveurs HAProxy fonctionnent en haute disponibilité grâce à Keepalived (protocole VRRP). debian_ha est le serveur principal (MASTER) et debian_ha2 le serveur de secours (BACKUP). Ils se partagent deux adresses IP virtuelles : 192.168.100.10, l'adresse du site pour les clients, et 192.168.200.254, la passerelle des serveurs web. Si debian_ha ou son service HAProxy tombe en panne, debian_ha2 récupère automatiquement ces adresses, de façon transparente pour les clients et les serveurs web.

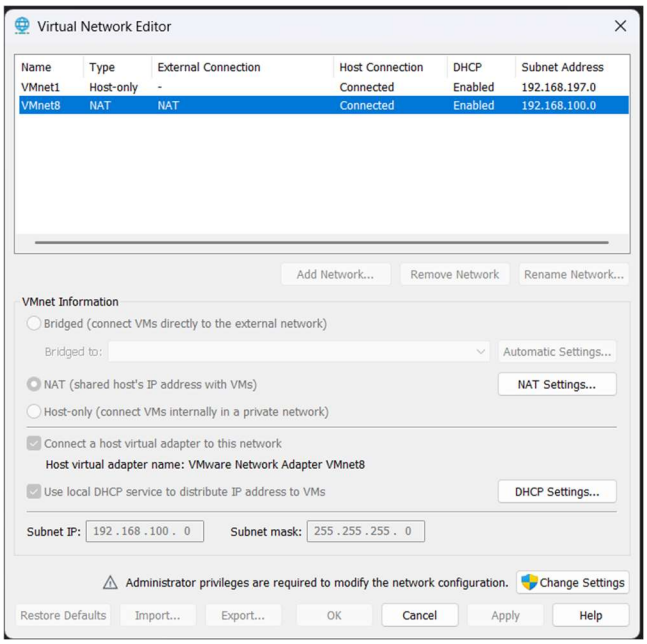
Enfin, HAProxy protège les serveurs web contre les attaques par saturation : il limite le nombre de requêtes par adresse IP, coupe les connexions trop lentes et plafonne le nombre de connexions envoyées à chaque serveur. Cette protection limite les effets d'une attaque DDoS, mais ne remplace pas un service spécialisé en cas d'attaque de grande ampleur.

Schéma du réseau :

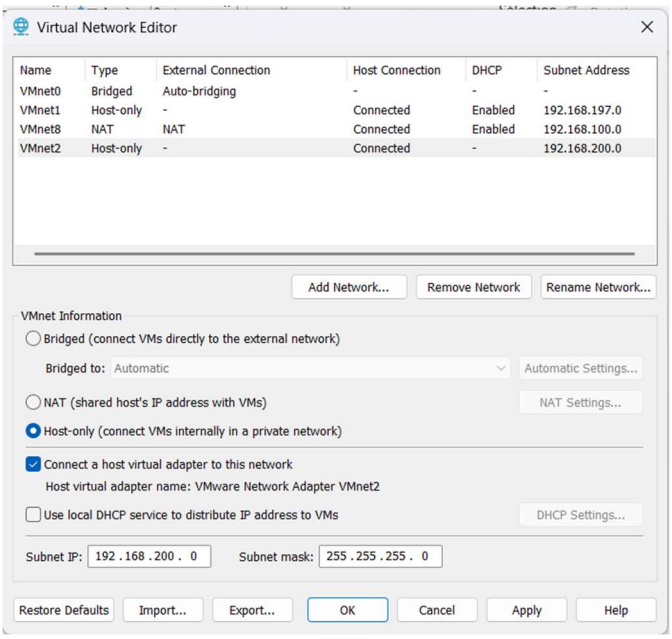


Le cadre en pointillés autour des deux proxys représente le groupe VRRP : les deux IP virtuelles sont portées par le MASTER (debian_ha) et passent automatiquement sur le BACKUP (debian_ha2) en cas de panne. Les clients et les serveurs web ne connaissent que ces deux VIP, et jamais les IP réelles des proxys.

Configuration VMnet8 : 192.168.100.0

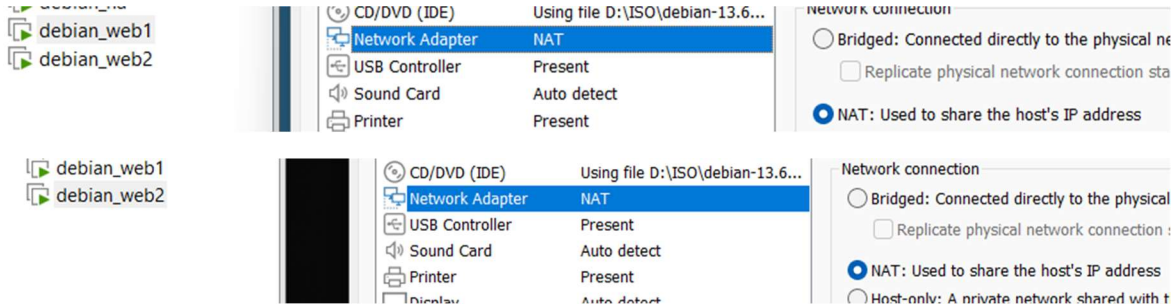


Configuration de VMnet2 : 192.168.200.0



Installation de apache2 et openssh-server sur debian_web1 et debian_web2

Pour le moment debian_web1 et debian_web2 sont toujours sur VMnet8 NAT :



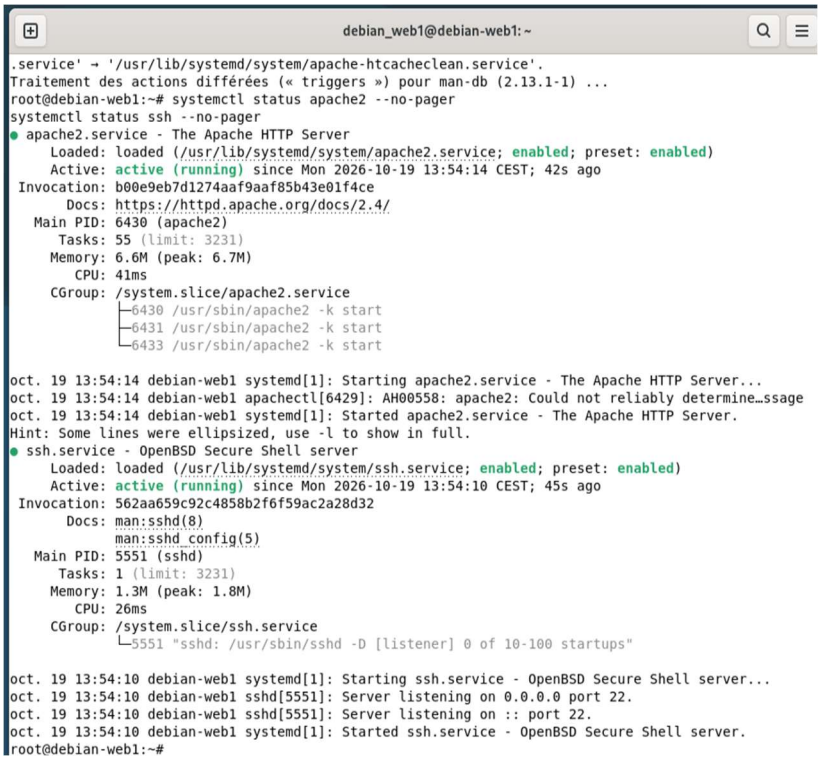
apt update

apt install -y apache2 openssh-server

Vérifier que les deux services tournent sur debian_web1 :

systemctl status apache2 --no-pager

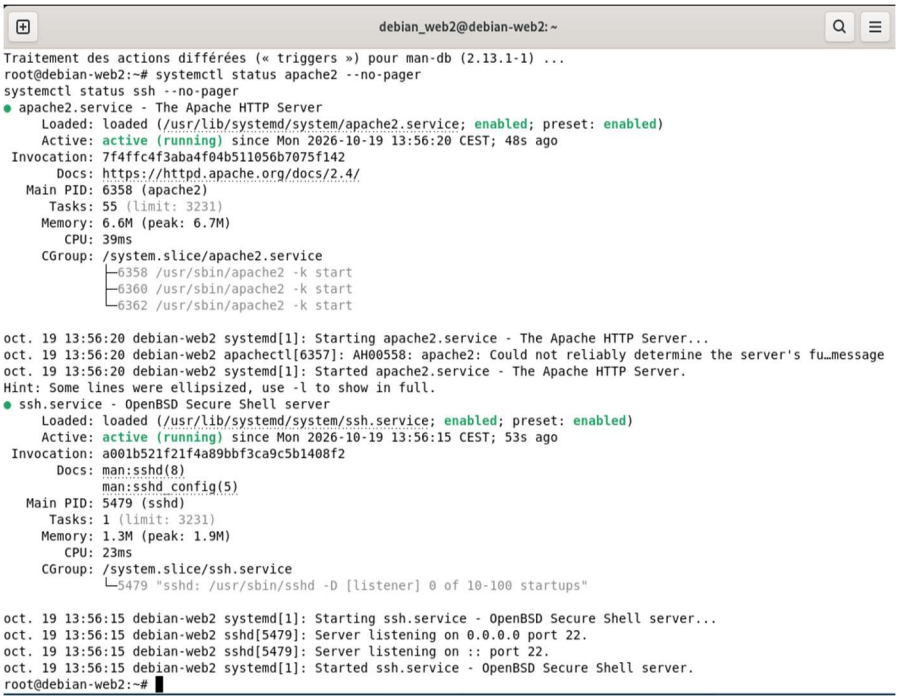
systemctl status ssh --no-pager



Vérifier que les deux services tournent sur debian_web2 :

systemctl status apache2 --no-pager

systemctl status ssh --no-pager



Un point important pour le SSH : par défaut, Debian interdit la connexion SSH en root avec un mot de passe. Le TP se connecte en root via PuTTY, on a deux options :

Se connecter avec un utilisateur normal, puis passer root avec su - ;

(Optionnel, non réalisé) Autoriser le root, dans un TP uniquement, jamais en production :

sed -i 's/^#\?PermitRootLogin.*/PermitRootLogin yes/' /etc/ssh/sshd_config

systemctl restart ssh

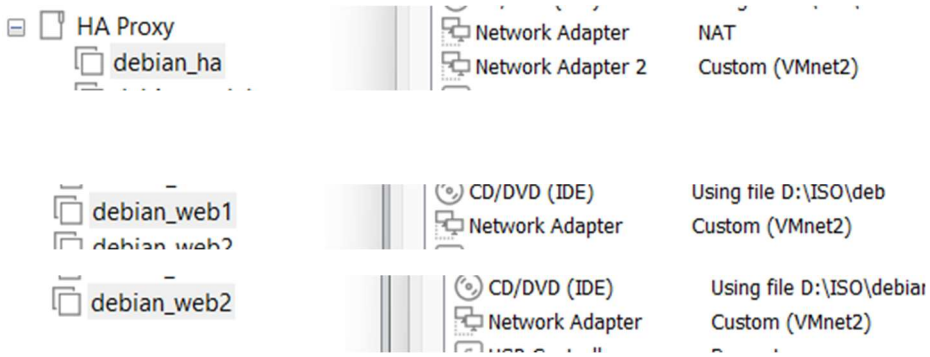
Installation d'HAproxy et iptables sur debian_ha :

apt install -y haproxy iptables

Une fois les paquets installés, éteindre les VMs

Ajouter la 2ème carte VMnet2 à debian_ha.

Changer la carte réseau de web1 et web2 sur VMnet2. (Ne pas supprimer VMnet8, juste changer en VMnet2)

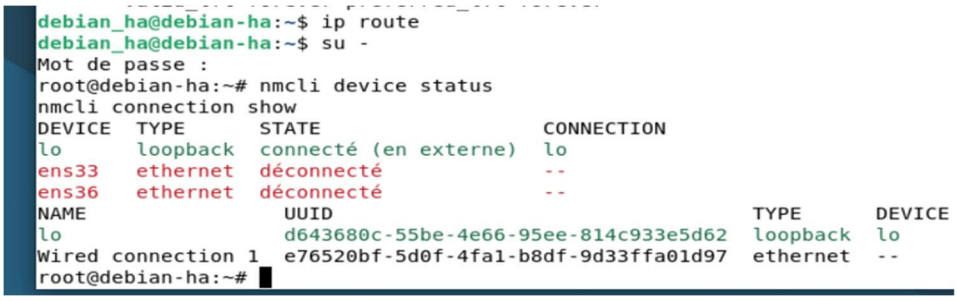


Après ce changement, web1 et web2 n'auront plus d'IP, puisqu'il n'y a pas de DHCP sur VMnet2. C'est normal : on leur donne une IP statique à l'étape suivante.

Passer en IP fixe sur debian_ha 192.168.100.10, passerelle 192.168.100.2 :

nmcli device status

nmcli connection show



nmcli connection delete "Wired connection 1"

nmcli connection add type ethernet ifname ens33 con-name wan \

ipv4.method manual ipv4.addresses 192.168.100.10/24 \

ipv4.gateway 192.168.100.2 ipv4.dns 192.168.100.2

nmcli connection add type ethernet ifname ens36 con-name lan \

ipv4.method manual ipv4.addresses 192.168.200.254/24

Activer les cartes réseaux :

nmcli connection up wan

nmcli connection up lan

Vérifications :

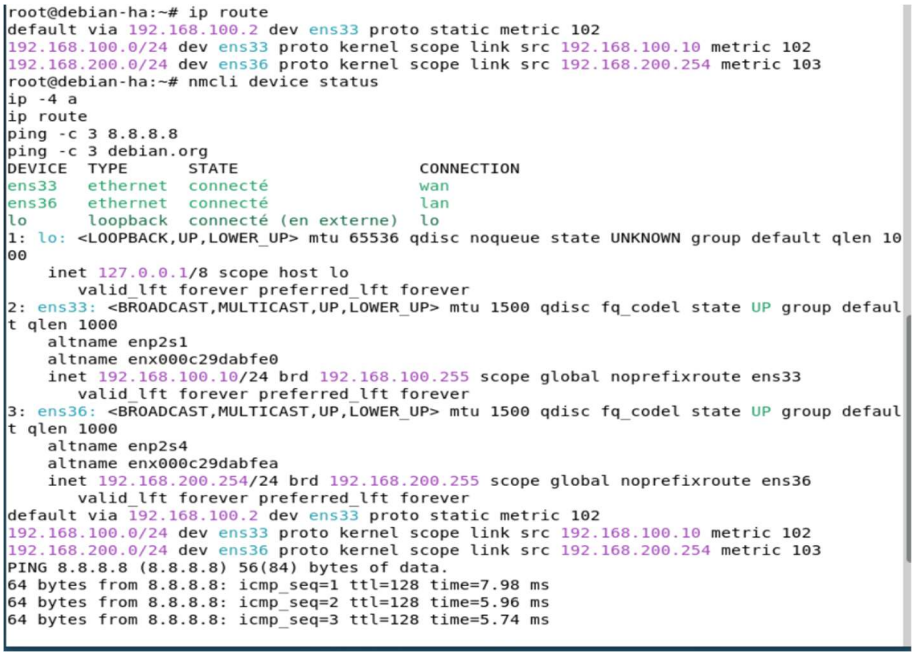
nmcli device status

ip -4 a

ip route

ping -c 3 8.8.8.8

ping -c 3 debian.org



```
--- 8.8.8.8 ping statistics ---
3 packets transmitted, 3 received, 0% packet loss, time 2003ms
rtt min/avg/max/mdev = 5.740/6.560/7.980/1.008 ms
PING debian.org (151.101.194.132) 56(84) bytes of data.
64 bytes from 151.101.194.132: icmp_seq=1 ttl=128 time=5.79 ms
64 bytes from 151.101.194.132: icmp_seq=2 ttl=128 time=6.51 ms
64 bytes from 151.101.194.132: icmp_seq=3 ttl=128 time=4.94 ms

--- debian.org ping statistics ---
3 packets transmitted, 3 received, 0% packet loss, time 2004ms
rtt min/avg/max/mdev = 4.941/5.748/6.514/0.642 ms
```

Attribution d'une ip fixe sur debian_web1 et debian_web2

Sur chaque VM web, vérifier d'abord le nom de la carte et le profil existant :

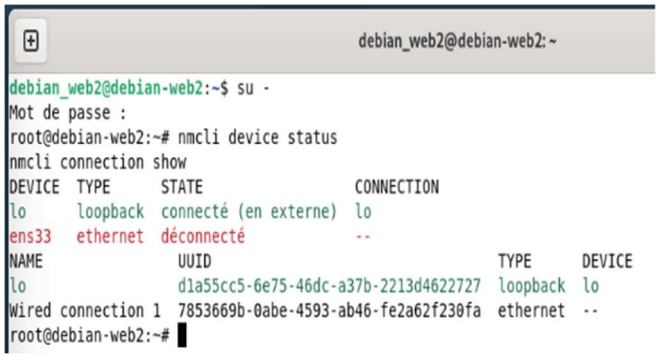
nmcli device status

nmcli connection show

Debian_web1



Debian_web2



Les deux serveurs sont dans la situation prévue : la carte s'appelle bien ens33, et « Wired connection 1 » essaie d'obtenir une IP par DHCP sur web1 (« obtention de la configuration IP »), ce qui n'aboutira jamais puisque VMnet2 n'a pas de DHCP. Sur web2, il n'est même pas attaché. On peut donc lancer les commandes telles quelles.

debian_web1 : Ip fixe 192.168.200.11, avec passerelle 192.168.200.254

nmcli connection delete "Wired connection 1"

nmcli connection add type ethernet ifname ens33 con-name lan \

ipv4.method manual ipv4.addresses 192.168.200.11/24 \

ipv4.gateway 192.168.200.254 ipv4.dns 8.8.8.8

nmcli connection up lan

```
root@debian-web1:~# nmcli connection delete "Wired connection 1"
nmcli connection add type ethernet ifname ens33 con-name lan \
  ipv4.method manual ipv4.addresses 192.168.200.11/24 \
  ipv4.gateway 192.168.200.254 ipv4.dns 8.8.8.8
nmcli connection up lan
Connexion « Wired connection 1 » (5e122b05-5328-4e79-ba9b-24976b352994) supprimée.
Connexion « lan » (f3c4fec6-86ed-40ac-a6a3-11f94874abd1) ajoutée avec succès.
Connexion activée (chemin D-Bus actif : /org/freedesktop/NetworkManager/ActiveConnection/11)
root@debian-web1:~# nmcli device status
DEVICE  TYPE      STATE                 CONNECTION
ens33   ethernet  connecté              lan
lo      loopback  connecté (en externe)  lo
root@debian-web1:~# ip route
default via 192.168.200.254 dev ens33 proto static metric 100
192.168.200.0/24 dev ens33 proto kernel scope link src 192.168.200.11 metric 100
root@debian-web1:~#
```

debian_web2 : ip fixe 192.168.200.12, passerelle 192.168.200.254

nmcli connection delete "Wired connection 1"

nmcli connection add type ethernet ifname ens33 con-name lan \

ipv4.method manual ipv4.addresses 192.168.200.12/24 \

ipv4.gateway 192.168.200.254 ipv4.dns 8.8.8.8

nmcli connection up lan

```
nmcli connection up lan
Connexion « Wired connection 1 » (7853669b-0abe-4593-ab46-fe2a62f230fa) supprimée.
Connexion « lan » (b252b077-9502-42f2-b938-395761c3cf02) ajoutée avec succès.
Connexion activée (chemin D-Bus actif : /org/freedesktop/NetworkManager/ActiveConnection/13)
root@debian-web2:~# nmcli device status
DEVICE  TYPE      STATE                 CONNECTION
ens33   ethernet  connecté              lan
lo      loopback  connecté (en externe)  lo
root@debian-web2:~# ip route
default via 192.168.200.254 dev ens33 proto static metric 100
192.168.200.0/24 dev ens33 proto kernel scope link src 192.168.200.12 metric 100
root@debian-web2:~#
```

Les deux serveurs web sont correctement configurés : ens33 → lan, IP .11 et .12, passerelle 192.168.200.254.

Test sur debian_ha :

ping -c 2 192.168.200.11

ping -c 2 192.168.200.12

```
--- 192.168.200.11 ping statistics ---
2 packets transmitted, 2 received, 0% packet loss, time 1008ms
rtt min/avg/max/mdev = 0.312/0.354/0.396/0.042 ms
PING 192.168.200.12 (192.168.200.12) 56(84) bytes of data.
64 bytes from 192.168.200.12: icmp_seq=1 ttl=64 time=0.313 ms
64 bytes from 192.168.200.12: icmp_seq=2 ttl=64 time=1.02 ms

--- 192.168.200.12 ping statistics ---
2 packets transmitted, 2 received, 0% packet loss, time 1020ms
rtt min/avg/max/mdev = 0.313/0.668/1.024/0.355 ms
root@debian-ha:~#
```

Debian_ha arrive à ping debian_web1 et debian_web2.

Création d'un serveur web sur debian_web1 :

cat > /var/www/html/index.html <<'EOF'

<!DOCTYPE html>

<html>

<body>

<h1>Serveur 1</h1>

Ceci est le serveur web 1.

</body>

</html>

EOF

```
root@debian-web1:~# cat > /var/www/html/index.html <<'EOF'
<!DOCTYPE html>
<html>
<body>
<h1>Serveur 1</h1>
<p>Ceci est le serveur web 1.</p>
</body>
</html>
EOF
root@debian-web1:~#
```

Création d'un serveur web sur debian_web2 :

cat > /var/www/html/index.html <<'EOF'

<!DOCTYPE html>

<html>

<body>

<h1>Serveur 2</h1>

Ceci est le serveur web 2.

</body>

</html>

EOF

```
root@debian-web2:~# cat > /var/www/html/index.html <<'EOF'
<!DOCTYPE html>
<html>
<body>
<h1>Serveur 2</h1>
<p>Ceci est le serveur web 2.</p>
</body>
</html>
EOF
root@debian-web2:~#
```

Vérification depuis debian_ha :

curl est un outil en ligne de commande qui envoie une requête HTTP à une adresse et affiche la réponse brute dans le terminal. C'est en quelque sorte un navigateur sans interface graphique : il récupère le code HTML de la page sans l'afficher mis en forme.

apt install -y curl

curl http://192.168.200.11

curl http://192.168.200.12

```
root@debian-ha:~# curl http://192.168.200.11
curl http://192.168.200.12
<!DOCTYPE html>
<html>
<body>
<h1>Serveur 1</h1>
<p>Ceci est le serveur web 1.</p>
</body>
</html>
<!DOCTYPE html>
<html>
<body>
<h1>Serveur 2</h1>
<p>Ceci est le serveur web 2.</p>
</body>
</html>
root@debian-ha:~#
```

Les deux serveurs répondent avec leur propre page, donc debian_ha peut les joindre sur le port 80.

Configuration de HAProxy sur debian_ha :

apt install -y haproxy *(Il était déjà installé, on ne refait pas la commande)*

Configuration du load balancing et de la page de statistiques :

Ajouter le bloc à la fin du fichier, avec bien >> et non > :

cat >> /etc/haproxy/haproxy.cfg <<'EOF'

```
listen cluster_web
   bind 192.168.100.10:80
   mode http
   balance roundrobin
   option forwardfor

   server web1 192.168.200.11:80 check
   server web2 192.168.200.12:80 check

   stats enable
   stats hide-version
   stats refresh 30s
   stats show-node
   stats auth admin:admin
   stats uri /stats
EOF
```

Vérifier la syntaxe, puis relancer le service HAproxy :

haproxy -c -f /etc/haproxy/haproxy.cfg

systemctl restart haproxy

systemctl status haproxy --no-pager

```
root@debian-ha:~# haproxy -c -f /etc/haproxy/haproxy.cfg
systemctl restart haproxy
systemctl status haproxy --no-pager
● haproxy.service - HAProxy Load Balancer
     Loaded: loaded (/usr/lib/systemd/system/haproxy.service; enabled; preset: enabled)
     Active: active (running) since Mon 2026-10-19 14:47:12 CEST; 25ms ago
 Invocation: fba91783a81845bf9a14185cfcd5d37a
       Docs: man:haproxy(1)
             file:/usr/share/doc/haproxy/configuration.txt.gz
   Main PID: 3327 (haproxy)
     Status: "Ready."
      Tasks: 2 (limit: 3231)
     Memory: 53.3M (peak: 53.6M)
        CPU: 65ms
     CGroup: /system.slice/haproxy.service
             ├─3327 /usr/sbin/haproxy -Ws -f /etc/haproxy/haproxy.cfg -p /run/haproxy.pid -S /run/haproxy-mas…
             └─3329 /usr/sbin/haproxy -Ws -f /etc/haproxy/haproxy.cfg -p /run/haproxy.pid -S /run/haproxy-mas…

oct. 19 14:47:12 debian-ha systemd[1]: Starting haproxy.service - HAProxy Load Balancer...
oct. 19 14:47:12 debian-ha haproxy[3327]: [NOTICE]   (3327) : New worker (3329) forked
oct. 19 14:47:12 debian-ha haproxy[3327]: [NOTICE]   (3327) : Loading success.
oct. 19 14:47:12 debian-ha systemd[1]: Started haproxy.service - HAProxy Load Balancer.
root@debian-ha:~#
```

Premier test depuis debian_ha :

for i in 1 2 3 4; do curl -s http://192.168.100.10 | grep h1; done

Cette boucle lance quatre requêtes vers HAProxy et n'affiche que le titre de chaque page. Le résultat doit alterner : Serveur 1, Serveur 2, Serveur 1, Serveur 2.

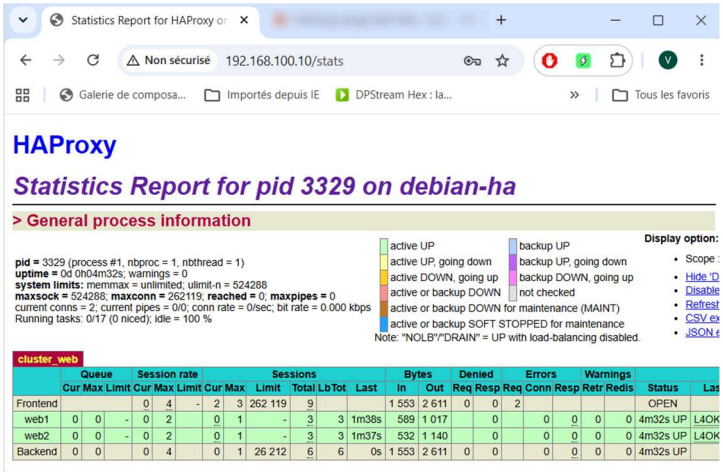
```
root@debian-ha:~# for i in 1 2 3 4; do curl -s http://192.168.100.10 | grep h1; done
<h1>Serveur 1</h1>
<h1>Serveur 2</h1>
<h1>Serveur 1</h1>
<h1>Serveur 2</h1>
```

Test depuis le PC hôte Windows 11 :

http://192.168.100.10



http://192.168.100.10/stats

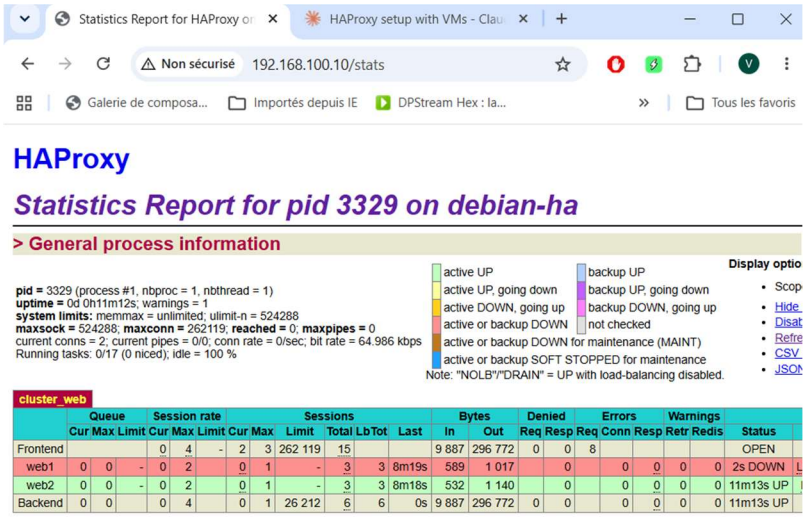


Test : Simuler une panne sur le serveur web

Sur debian_web1 :

systemctl stop apache2

Rafraîchir la page de stats : web1 doit passer en rouge (DOWN).



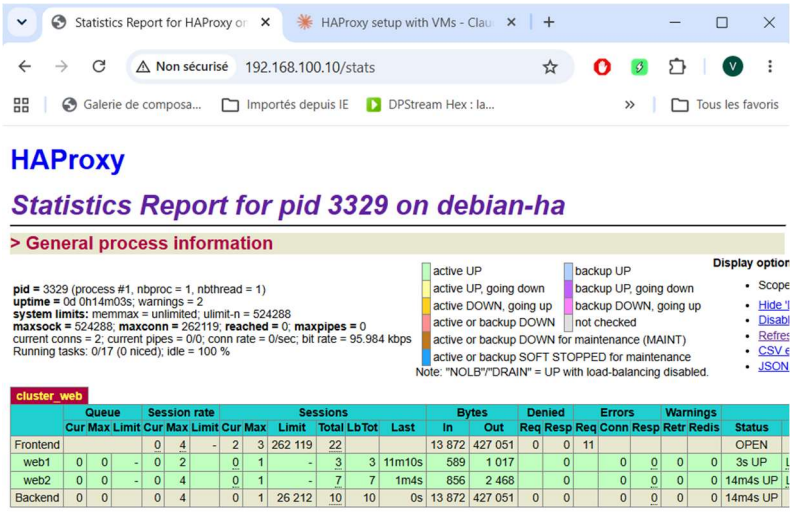
Refaire le test depuis debian_ha :

for i in 1 2 3 4; do curl -s http://192.168.100.10 | grep h1; done

```
root@debian-ha:~# for i in 1 2 3 4; do curl -s http://192.168.100.10 | grep h1; done
<h1>Serveur 2</h1>
<h1>Serveur 2</h1>
<h1>Serveur 2</h1>
<h1>Serveur 2</h1>
root@debian-ha:~#
```

Remettre web1 en service :

systemctl start apache2



Le serveur web de debian_web1 est reparti.

La partie load balancing et tolérance de panne est terminée.

Partie 2 : debian_ha en routeur (iptables)

Elle a deux objectifs : donner Internet à web1/web2 via debian_ha, et permettre le SSH vers web1/web2 depuis votre PC grâce aux ports 2221/2222.

Sur debian_ha :

apt install -y iptables

Activation du routage et autorisation du SSH

cat > /etc/init.d/firewall <<'EOF'

#!/bin/sh

BEGIN INIT INFO

```
# Provides:        firewall
# Required-Start:    $network
# Required-Stop:     $network
# Default-Start:     2 3 4 5
# Default-Stop:      0 1 6
# Short-Description: Regles iptables du TP HAProxy
### END INIT INFO

WAN=ens33
WAN_IP=192.168.100.10
WEB1=192.168.200.11
WEB2=192.168.200.12

start() {
   echo -n "Application des regles IpTables: "
   iptables -F
   iptables -X
   iptables -t nat -F

   # Activation du routage
   sysctl -w net.ipv4.ip_forward=1

   # Par defaut, on refuse le forwarding
   iptables -P FORWARD DROP

   # On autorise le forwarding seulement pour les serveurs web
   iptables -A FORWARD -d $WEB1 -j ACCEPT
   iptables -A FORWARD -s $WEB1 -j ACCEPT
   iptables -A FORWARD -d $WEB2 -j ACCEPT
   iptables -A FORWARD -s $WEB2 -j ACCEPT
```

```
    # NAT du reseau prive vers l'exterieur
    iptables -t nat -A POSTROUTING -o $WAN -j MASQUERADE

    # Redirections SSH vers les serveurs web
    iptables -t nat -A PREROUTING -p tcp -d $WAN_IP --dport 2221 -j DNAT --to-destination $WEB1:22
    iptables -t nat -A PREROUTING -p tcp -d $WAN_IP --dport 2222 -j DNAT --to-destination $WEB2:22

    echo "termine"
}

stop() {
    echo -n "Suppression des regles IpTables: "
    iptables -F
    iptables -X
    iptables -t nat -F
    iptables -P FORWARD ACCEPT
    sysctl -w net.ipv4.ip_forward=0
    echo "termine"
}

case "$1" in
    start)   start ;;
    stop)    stop ;;
    restart) stop; start ;;
    status)
        iptables -L -n -v
        iptables -t nat -L -n -v
        ;;
    *)
        echo "Usage: firewall {start|stop|restart|status}"
        ;;
```

esac

exit 0

EOF

chmod +x /etc/init.d/firewall

systemctl daemon-reload

service firewall start

Pour que les règles soient appliquées automatiquement au démarrage :

systemctl enable firewall

Vérifications

Sur debian_ha :

```
root@debian-ha:~# service firewall status
● firewall.service - LSB: Regles iptables du TP HAProxy
     Loaded: loaded (/etc/init.d/firewall; generated)
     Active: active (exited) since Mon 2026-10-19 15:45:30 CEST; 2min 3s ago
 Invocation: 3c15e9502bb94c66ae79a8d7e3a96168
       Docs: man:systemd-sysv-generator(8)
   Mem peak: 1.7M
        CPU: 31ms

oct. 19 15:45:29 debian-ha systemd[1]: Starting firewall.service - LSB: Regles iptables du TP HAProxy...
oct. 19 15:45:29 debian-ha firewall[3974]: Application des regles IpTables:
oct. 19 15:45:29 debian-ha firewall[3980]: net.ipv4.ip_forward = 1
oct. 19 15:45:30 debian-ha firewall[3974]: termine
oct. 19 15:45:30 debian-ha systemd[1]: Started firewall.service - LSB: Regles iptables du TP HAProxy.
root@debian-ha:~#
```

Tests sur web1 et web2 :

ping -c 3 8.8.8.8

ping -c 3 debian.org

apt update

```
root@debian-web1:~# ping -c 3 8.8.8.8
ping -c 3 debian.org
apt update
PING 8.8.8.8 (8.8.8.8) 56(84) bytes of data.
64 bytes from 8.8.8.8: icmp_seq=1 ttl=127 time=6.01 ms
64 bytes from 8.8.8.8: icmp_seq=2 ttl=127 time=5.90 ms
64 bytes from 8.8.8.8: icmp_seq=3 ttl=127 time=6.86 ms

--- 8.8.8.8 ping statistics ---
3 packets transmitted, 3 received, 0% packet loss, time 2002ms
rtt min/avg/max/mdev = 5.899/6.257/6.861/0.429 ms
PING debian.org (151.101.2.132) 56(84) bytes of data.
64 bytes from 151.101.2.132: icmp_seq=1 ttl=127 time=5.68 ms
64 bytes from 151.101.2.132: icmp_seq=2 ttl=127 time=5.42 ms
64 bytes from 151.101.2.132: icmp_seq=3 ttl=127 time=6.33 ms

--- debian.org ping statistics ---
3 packets transmitted, 3 received, 0% packet loss, time 2004ms
rtt min/avg/max/mdev = 5.420/5.808/6.328/0.382 ms
Atteint : 1 http://security.debian.org/debian-security trixie-secu
Atteint : 2 http://deb.debian.org/debian trixie InRelease
Atteint : 3 http://deb.debian.org/debian trixie-updates InRelease
Tous les paquets sont à jour.
root@debian-web1:~#
```

```
root@debian-web2:~# ping -c 3 8.8.8.8
ping -c 3 debian.org
apt update
PING 8.8.8.8 (8.8.8.8) 56(84) bytes of data.
64 bytes from 8.8.8.8: icmp_seq=1 ttl=127 time=5.61 ms
64 bytes from 8.8.8.8: icmp_seq=2 ttl=127 time=7.08 ms
64 bytes from 8.8.8.8: icmp_seq=3 ttl=127 time=6.15 ms

--- 8.8.8.8 ping statistics ---
3 packets transmitted, 3 received, 0% packet loss, time 2004ms
rtt min/avg/max/mdev = 5.605/6.278/7.082/0.609 ms
PING debian.org (151.101.2.132) 56(84) bytes of data.
64 bytes from 151.101.2.132: icmp_seq=1 ttl=127 time=5.31 ms
64 bytes from 151.101.2.132: icmp_seq=2 ttl=127 time=5.59 ms
64 bytes from 151.101.2.132: icmp_seq=3 ttl=127 time=7.06 ms

--- debian.org ping statistics ---
3 packets transmitted, 3 received, 0% packet loss, time 2004ms
rtt min/avg/max/mdev = 5.310/5.985/7.057/0.766 ms
Atteint : 1 http://security.debian.org/debian-security trixie-secu
Atteint : 2 http://deb.debian.org/debian trixie InRelease
Atteint : 3 http://deb.debian.org/debian trixie-updates InRelease
Tous les paquets sont à jour.
root@debian-web2:~#
```

Tests en SSH ou Putty depuis le PC hôte Windows :

Si SSH, d'abord passer en utilisateur normal, puis en root.

ssh debian_web1@192.168.100.10 -p 2221

ssh debian_web2@192.168.100.10 -p 2222

```
debian_web1@debian-web1: ~
This key is not known by any other names.
Are you sure you want to continue connecting (yes/no/[fingerprint])? yes
Warning: Permanently added '[192.168.100.10]:2221' (ED25519) to the list of known hosts.
debian_web1@192.168.100.10's password:
Linux debian-web1 6.12.113+deb13-amd64 #1 SMP PREEMPT_DYNAMIC Debian 6.12.113-1 (2026-10-15) x86_64

The programs included with the Debian GNU/Linux system are free software;
the exact distribution terms for each program are described in the
individual files in /usr/share/doc/*/copyright.

Debian GNU/Linux comes with ABSOLUTELY NO WARRANTY, to the extent
permitted by applicable law.
debian_web1@debian-web1:~$ su -
Mot de passe :
root@debian-web1:~# ip a
1: lo: <LOOPBACK,UP,LOWER_UP> mtu 65536 qdisc noqueue state UNKNOWN group default qlen 1000
    link/loopback 00:00:00:00:00:00 brd 00:00:00:00:00:00
    inet 127.0.0.1/8 scope host lo
       valid_lft forever preferred_lft forever
    inet6 ::1/128 scope host noprefixroute
       valid_lft forever preferred_lft forever
2: ens33: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc fq_codel state UP group default qlen 1000
    link/ether 00:0c:29:bb:c3:57 brd ff:ff:ff:ff:ff:ff
    altname enp2s1
    altname enx000c29bbc357
    inet 192.168.200.11/24 brd 192.168.200.255 scope global noprefixroute ens33
       valid_lft forever preferred_lft forever
    inet6 fe80::5c76:c8e4:a033:d2c7/64 scope link noprefixroute
       valid_lft forever preferred_lft forever
root@debian-web1:~#
```

```
debian_web2@debian-web2: ~

C:\Windows\System32>ssh debian_web2@192.168.100.10 -p 2222
The authenticity of host '[192.168.100.10]:2222 ([192.168.100.10]:2222)' can't be established.
ED25519 key fingerprint is SHA256:E78eKAsFbduCRoBiPXhyiWTYJERzclAGl5E41Aykur8.
This key is not known by any other names.
Are you sure you want to continue connecting (yes/no/[fingerprint])? yes
Warning: Permanently added '[192.168.100.10]:2222' (ED25519) to the list of known hosts.
debian_web2@192.168.100.10's password:
Linux debian-web2 6.12.113+deb13-amd64 #1 SMP PREEMPT_DYNAMIC Debian 6.12.113-1 (2026-10-15) x86_64

The programs included with the Debian GNU/Linux system are free software;
the exact distribution terms for each program are described in the
individual files in /usr/share/doc/*/copyright.

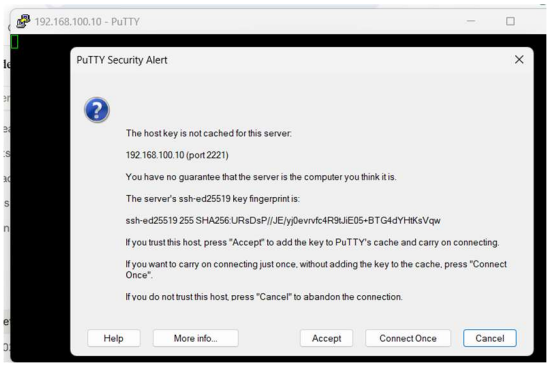
Debian GNU/Linux comes with ABSOLUTELY NO WARRANTY, to the extent
permitted by applicable law.
debian_web2@debian-web2:~$ su -
Mot de passe :
root@debian-web2:~# ip a
1: lo: <LOOPBACK,UP,LOWER_UP> mtu 65536 qdisc noqueue state UNKNOWN group default qlen 1000
    link/loopback 00:00:00:00:00:00 brd 00:00:00:00:00:00
    inet 127.0.0.1/8 scope host lo
       valid_lft forever preferred_lft forever
    inet6 ::1/128 scope host noprefixroute
       valid_lft forever preferred_lft forever
2: ens33: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc fq_codel state UP group default qlen 1000
    link/ether 00:0c:29:54:e9:a0 brd ff:ff:ff:ff:ff:ff
    altname enp2s1
    altname enx000c2954e9a0
    inet 192.168.200.12/24 brd 192.168.200.255 scope global noprefixroute ens33
       valid_lft forever preferred_lft forever
    inet6 fe80::d26a:abb2:8a0e:181d/64 scope link noprefixroute
       valid_lft forever preferred_lft forever
root@debian-web2:~#
```

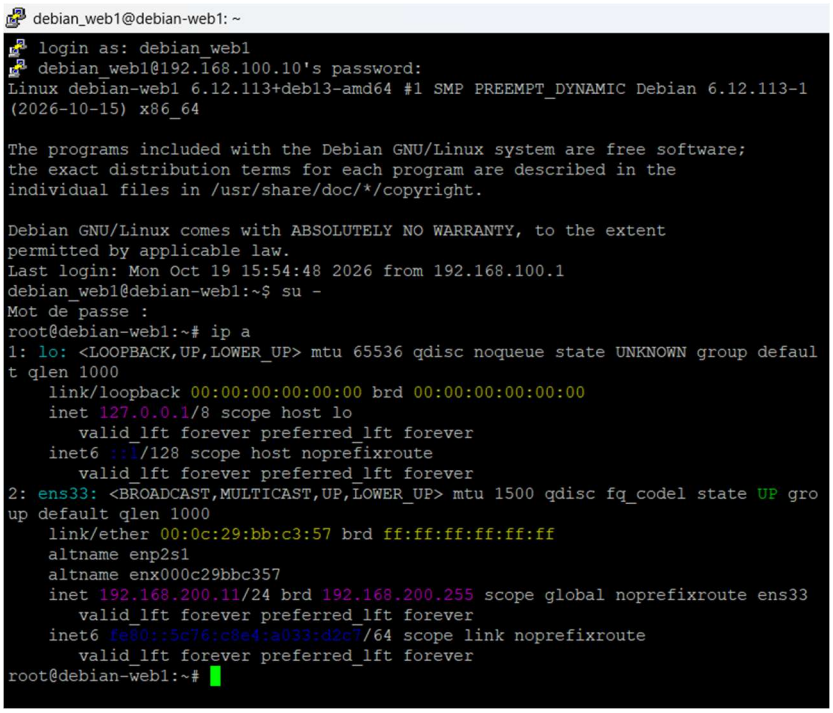
PuTTY fonctionne aussi : hôte 192.168.100.10, port 2221 ou 2222. Le prompt doit indiquer debian-web1 ou debian-web2 alors que vous vous êtes connecté à l'IP du proxy, ce qui prouve que la redirection fonctionne.

Test Putty :

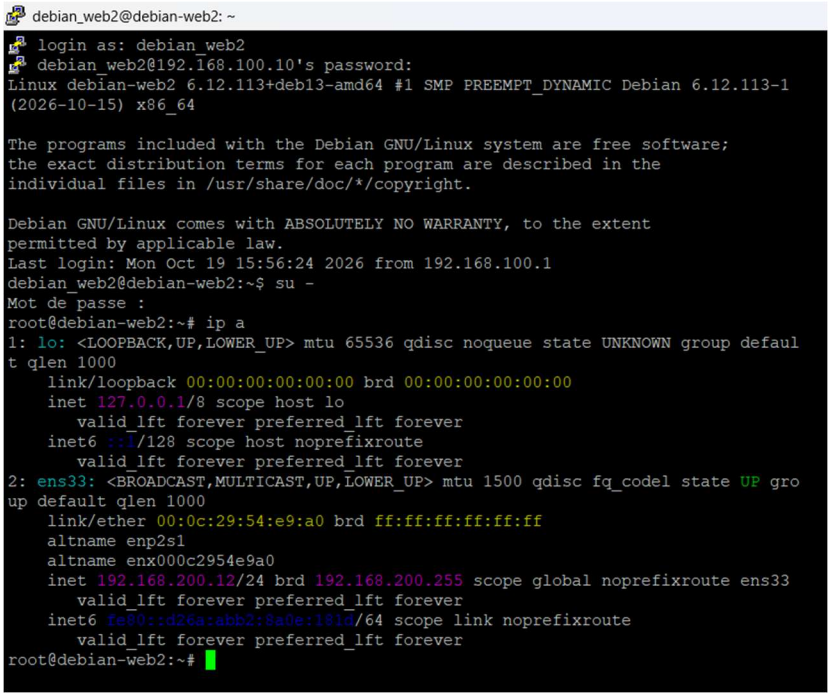
Pour debian_web1, 192.168.100.10, Port : 2221

Login : debian_web1





Pour debian_web2, 192.168.100.10, Port : 2222



Toutes les parties fonctionnent :

Load balancing : les requêtes alternent entre Serveur 1 et Serveur 2.

Supervision et panne : web1 est passé DOWN puis a été réintégré automatiquement.

Routage et NAT : web1/web2 ont Internet via debian_ha (ping, apt update).

Redirection SSH : via 192.168.100.10, le port 2221 mène à web1 et le port 2222 à web2. Les ip a le confirment : on est sur 192.168.200.11 et 192.168.200.12, alors que l'on est connecté à l'IP du proxy.

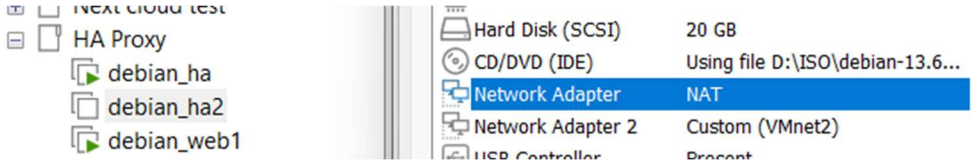
Ajout d’un serveur HAproxy debian_ha2 de redondance :

Création d’une VM debian_ha2

Dans les paramètres de debian_ha2, ajout des deux cartes, comme sur debian_ha :

Network Adapter : NAT ; (VMnet8)

Network Adapter 2 : Custom (VMnet2)



Configuration réseau

Vérification des noms des cartes :

nmcli device status

nmcli connection show

```
root@debian-ha2:~# nmcli device status
nmcli connection show
DEVICE  TYPE      STATE                   CONNECTION
lo      loopback  connecté (en externe)  lo
ens33   ethernet  déconnecté              --
ens36   ethernet  déconnecté              --
NAME                UUID                                  TYPE      DEVICE
lo                  460efbae-0e18-4f35-a2f2-bdea8696cbc2  loopback  lo
Wired connection 1  4f128882-dc36-4168-8764-920c74c50855  ethernet  --
root@debian-ha2:~#
```

Ajout d’une ip fixe pour debian_ha2 : 192.168.100.12, passerelle 192.168.100.2 et activation des cartes réseaux :

nmcli connection delete "Wired connection 1"

nmcli connection add type ethernet ifname ens33 con-name wan \

ipv4.method manual ipv4.addresses 192.168.100.12/24 \

ipv4.gateway 192.168.100.2 ipv4.dns 192.168.100.2

nmcli connection add type ethernet ifname ens36 con-name lan \

ipv4.method manual ipv4.addresses 192.168.200.253/24

nmcli connection up wan

nmcli connection up lan

Vérifications :

nmcli device status

nmcli connection show

ip a

ip route

```
root@debian-ha2:~# nmcli device status
nmcli connection show
DEVICE  TYPE      STATE                   CONNECTION
ens33   ethernet  connecté                wan
ens36   ethernet  connecté                lan
lo      loopback  connecté (en externe)  lo
NAME  UUID                                  TYPE      DEVICE
wan   78c10518-9b05-404a-b092-a386c5a5fed3  ethernet  ens33
lan   cd83f7cc-26d6-487b-86f2-973cf2c3aff5  ethernet  ens36
lo    460efbae-0e18-4f35-a2f2-bdea8696cbc2  loopback  lo
root@debian-ha2:~# ip a
1: lo: <LOOPBACK,UP,LOWER_UP> mtu 65536 qdisc noqueue state UNKNOWN group default qlen 1000
    link/loopback 00:00:00:00:00:00 brd 00:00:00:00:00:00
    inet 127.0.0.1/8 scope host lo
       valid_lft forever preferred_lft forever
    inet6 ::1/128 scope host noprefixroute
       valid_lft forever preferred_lft forever
2: ens33: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc fq_codel state UP group default qlen 100
0
    link/ether 00:0c:29:2e:d6:41 brd ff:ff:ff:ff:ff:ff
    altname enp2s1
    altname enx000c292ed641
    inet 192.168.100.12/24 brd 192.168.100.255 scope global noprefixroute ens33
       valid_lft forever preferred_lft forever
    inet6 fe80::9e1f:5acf:4c9f:272d/64 scope link noprefixroute
       valid_lft forever preferred_lft forever
3: ens36: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc fq_codel state UP group default qlen 100
0
    link/ether 00:0c:29:2e:d6:4b brd ff:ff:ff:ff:ff:ff
    altname enp2s4
    altname enx000c292ed64b
    inet 192.168.200.253/24 brd 192.168.200.255 scope global noprefixroute ens36
       valid_lft forever preferred_lft forever
    inet6 fe80::660d:a1af:bfed:cb07/64 scope link noprefixroute
       valid_lft forever preferred_lft forever
root@debian-ha2:~# ip route
```

```
root@debian-ha2:~# ip route
default via 192.168.100.2 dev ens33 proto static metric 102
192.168.100.0/24 dev ens33 proto kernel scope link src 192.168.100.12 metric 102
192.168.200.0/24 dev ens36 proto kernel scope link src 192.168.200.253 metric 103
root@debian-ha2:~#
```

Installation des paquets :

apt update

apt install -y haproxy iptables keepalived curl

On installe dès maintenant keepalived, l'outil qui gérera les IP virtuelles. On le configurera à la fin. Keepalived est un logiciel libre pour Linux qui assure la haute disponibilité d'une adresse IP. Il fait en sorte qu'une adresse reste toujours joignable, même si la machine qui la porte tombe en panne.

Autoriser l'écoute sur une IP virtuelle

HAProxy doit écouter sur 192.168.100.10, une adresse que debian_ha2 ne possédera que lorsqu'elle sera MASTER. Par défaut, Linux refuse de démarrer un service sur une IP qu'il n'a pas. Ce réglage l'autorise :

echo "net.ipv4.ip_nonlocal_bind=1" > /etc/sysctl.d/99-haproxy-vip.conf

sysctl --system

```
root@debian-ha2:~# echo "net.ipv4.ip_nonlocal_bind=1" > /etc/sysctl.d/99-haproxy-vip.conf
sysctl --system
* Applique /usr/lib/sysctl.d/10-coredump-debian.conf …
* Applique /usr/lib/sysctl.d/30-localsearch.conf …
* Applique /usr/lib/sysctl.d/50-bubblewrap.conf …
* Applique /usr/lib/sysctl.d/50-default.conf …
* Applique /usr/lib/sysctl.d/50-pid-max.conf …
* Applique /etc/sysctl.d/99-haproxy-vip.conf …
kernel.core_pattern = core
fs.inotify.max_user_watches = 65536
kernel.unprivileged_userns_clone = 1
kernel.sysrq = 0x01b6
kernel.core_uses_pid = 1
net.ipv4.conf.default.rp_filter = 2
net.ipv4.conf.ens33.rp_filter = 2
net.ipv4.conf.ens36.rp_filter = 2
net.ipv4.conf.lo.rp_filter = 2
net.ipv4.conf.default.accept_source_route = 0
net.ipv4.conf.ens33.accept_source_route = 0
net.ipv4.conf.ens36.accept_source_route = 0
net.ipv4.conf.lo.accept_source_route = 0
net.ipv4.conf.default.promote_secondaries = 1
net.ipv4.conf.ens33.promote_secondaries = 1
net.ipv4.conf.ens36.promote_secondaries = 1
net.ipv4.conf.lo.promote_secondaries = 1
net.ipv4.ping_group_range = 0 2147483647
net.core.default_qdisc = fq_codel
fs.protected_hardlinks = 1
fs.protected_symlinks = 1
fs.protected_regular = 2
fs.protected_fifos = 1
vm.max_map_count = 1048576
kernel.pid_max = 4194304
net.ipv4.ip_nonlocal_bind = 1
root@debian-ha2:~#
```

On fera la même chose sur debian_ha plus tard.

Copie de la configuration HAProxy de debian_ha vers debian_ha2 :

cat >> /etc/haproxy/haproxy.cfg <<'EOF'

listen cluster_web

bind 192.168.100.10:80

mode http

balance roundrobin

option forwardfor

server web1 192.168.200.11:80 check

server web2 192.168.200.12:80 check

stats enable

stats hide-version

stats refresh 30s

stats show-node

stats auth admin:admin

```
    stats uri /stats
EOF

haproxy -c -f /etc/haproxy/haproxy.cfg
systemctl restart haproxy
```

La configuration est identique à celle de debian_ha. L'adresse 192.168.100.10 reste celle du site, elle deviendra simplement virtuelle.

Copie du script firewall de debian_ha sur debian_ha2 :

```
cat > /etc/init.d/firewall <<'EOF'
#!/bin/sh
### BEGIN INIT INFO
# Provides:        firewall
# Required-Start:    $network
# Required-Stop:     $network
# Default-Start:     2 3 4 5
# Default-Stop:      0 1 6
# Short-Description: Regles iptables du TP HAProxy
### END INIT INFO

WAN=ens33
WAN_IP=192.168.100.10
WEB1=192.168.200.11
WEB2=192.168.200.12

start() {
    echo -n "Application des regles IpTables: "
    iptables -F
    iptables -X
    iptables -t nat -F
```

```
    sysctl -w net.ipv4.ip_forward=1
    iptables -P FORWARD DROP
    iptables -A FORWARD -d $WEB1 -j ACCEPT
    iptables -A FORWARD -s $WEB1 -j ACCEPT
    iptables -A FORWARD -d $WEB2 -j ACCEPT
    iptables -A FORWARD -s $WEB2 -j ACCEPT
    iptables -t nat -A POSTROUTING -o $WAN -j MASQUERADE
    iptables -t nat -A PREROUTING -p tcp -d $WAN_IP --dport 2221 -j DNAT --to-destination $WEB1:22
    iptables -t nat -A PREROUTING -p tcp -d $WAN_IP --dport 2222 -j DNAT --to-destination $WEB2:22
    echo "termine"
}

stop() {
    echo -n "Suppression des regles IpTables: "
    iptables -F
    iptables -X
    iptables -t nat -F
    iptables -P FORWARD ACCEPT
    sysctl -w net.ipv4.ip_forward=0
    echo "termine"
}

case "$1" in
    start)   start ;;
    stop)    stop ;;
    restart) stop; start ;;
    status)
        iptables -L -n -v
        iptables -t nat -L -n -v
        ;;
    *)
```

```
    echo "Usage: firewall {start|stop|restart|status}"
    ;;
esac
exit 0
EOF
```

chmod +x /etc/init.d/firewall

systemctl daemon-reload

systemctl enable firewall

service firewall start

Vérifications :

systemctl status haproxy --no-pager

curl -s http://192.168.200.11 | grep h1

curl -s http://192.168.200.12 | grep h1

```
root@debian-ha2:~# systemctl status haproxy --no-pager
curl -s http://192.168.200.11 | grep h1
curl -s http://192.168.200.12 | grep h1
● haproxy.service - HAProxy Load Balancer
     Loaded: loaded (/usr/lib/systemd/system/haproxy.service; enabled; preset: enabled)
     Active: active (running) since Tue 2026-10-20 10:54:46 CEST; 1min 58s ago
 Invocation: 8af39cbff53b40399d1288b1b74ca1df
       Docs: man:haproxy(1)
             file:/usr/share/doc/haproxy/configuration.txt.gz
   Main PID: 3817 (haproxy)
     Status: "Ready."
      Tasks: 2 (limit: 2998)
     Memory: 53.8M (peak: 54M)
        CPU: 137ms
     CGroup: /system.slice/haproxy.service
             ├─3817 /usr/sbin/haproxy -Ws -f /etc/haproxy/haproxy.cfg -p /run/haproxy.pid -S /run/…
             └─3819 /usr/sbin/haproxy -Ws -f /etc/haproxy/haproxy.cfg -p /run/haproxy.pid -S /run/…

oct. 20 10:54:46 debian-ha2 systemd[1]: Starting haproxy.service - HAProxy Load Balancer...
oct. 20 10:54:46 debian-ha2 haproxy[3817]: [NOTICE]   (3817) : New worker (3819) forked
oct. 20 10:54:46 debian-ha2 haproxy[3817]: [NOTICE]   (3817) : Loading success.
oct. 20 10:54:46 debian-ha2 systemd[1]: Started haproxy.service - HAProxy Load Balancer.
<h1>Serveur 1</h1>
<h1>Serveur 2</h1>
root@debian-ha2:~#
```

Vérifie le statut d'HAproxy et le contenu des deux sites internet. À ce stade, debian_ha2 est prête mais inactive : elle ne possède pas encore 192.168.100.10, donc tout le trafic passe toujours par debian_ha. L'étape suivante est la configuration de Keepalived sur les deux proxys.

Autoriser l'IP virtuelle et installer Keepalived sur debian_ha

echo "net.ipv4.ip_nonlocal_bind=1" > /etc/sysctl.d/99-haproxy-vip.conf

sysctl --system

apt install -y keepalived

Écrire la configuration de Keepalived :

```
cat > /etc/keepalived/keepalived.conf <<'EOF'
global_defs {
   enable_script_security
   script_user root
}

vrrp_script chk_haproxy {
   script "/usr/bin/systemctl is-active --quiet haproxy"
   interval 2
   fall 2
   rise 2
}

vrrp_instance VI_1 {
   state MASTER
   interface ens36
   virtual_router_id 51
   priority 150
   advert_int 1

   authentication {
      auth_type PASS
      auth_pass tpha2026
   }

   virtual_ipaddress {
      192.168.100.10/24 dev ens33
      192.168.200.254/24 dev ens36
   }

   track_script {
```

```
    chk_haproxy
  }
}
EOF
```

Les éléments importants de ce fichier :

vrrp_script chk_haproxy vérifie toutes les 2 secondes que HAProxy tourne. Après 2 échecs, la machine cède les VIP.

state MASTER et priority 150 : c'est le proxy principal, la plus haute priorité gagne.

interface ens36 : les messages « je suis vivant » passent par le réseau interne.

virtual_router_id 51 identifie le groupe et doit être identique sur les deux proxys.

auth_pass est un mot de passe partagé entre les deux (8 caractères maximum).

virtual_ipaddress liste les deux VIP et l'interface qui porte chacune.

Changer les IP réelles de debian_ha 192.168.100.11 et 192.168.200.252

Les adresses 192.168.100.10 et 192.168.200.254 vont devenir des IP virtuelles (VIP), gérées par Keepalived. Une IP virtuelle ne doit appartenir de façon fixe à aucune machine : c'est Keepalived qui l'attribue au serveur MASTER et la transfère au BACKUP en cas de panne. On retire donc ces adresses de la configuration fixe de debian_ha, qui reçoit de nouvelles IP réelles (192.168.100.11 et 192.168.200.252).

On aurait pu choisir d'autres adresses comme VIP, par exemple 192.168.100.20 et 192.168.200.250, et laisser debian_ha garder les siennes. On a conservé les adresses d'origine car toute l'infrastructure est déjà configurée autour d'elles : la passerelle de debian_web1 et debian_web2, l'adresse d'écoute (bind) de HAProxy, les règles de redirection SSH du script firewall et l'adresse utilisée par les clients. Ainsi, ni les serveurs web ni les clients n'ont besoin d'être reconfigurés.

Pendant le changement d'adresses, le site est indisponible quelques secondes, le temps que Keepalived démarre et attribue les VIP.

```
nmcli connection modify wan ipv4.addresses 192.168.100.11/24
nmcli connection modify lan ipv4.addresses 192.168.200.252/24
nmcli connection up wan
nmcli connection up lan
```

```
root@debian-ha:~# nmcli connection modify wan ipv4.addresses 192.168.100.11/24
nmcli connection modify lan ipv4.addresses 192.168.200.252/24
nmcli connection up wan
nmcli connection up lan
Connexion activée (chemin D-Bus actif : /org/freedesktop/NetworkManager/ActiveConnection/18)
Connexion activée (chemin D-Bus actif : /org/freedesktop/NetworkManager/ActiveConnection/19)
root@debian-ha:~#
```

Démarrer Keepalived et relancer HAProxy :

systemctl enable --now keepalived

systemctl restart keepalived

systemctl restart haproxy

Vérification :

ip -4 a

```
root@debian-ha:~# ip -4 a
1: lo: <LOOPBACK,UP,LOWER_UP> mtu 65536 qdisc noqueue state UNKNOWN group default qlen 1000
    inet 127.0.0.1/8 scope host lo
       valid_lft forever preferred_lft forever
2: ens33: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc fq_codel state UP group default qlen 1000
    altname enp2s1
    altname enx000c29dabfe0
    inet 192.168.100.11/24 brd 192.168.100.255 scope global noprefixroute ens33
       valid_lft forever preferred_lft forever
    inet 192.168.100.10/24 scope global secondary proto keepalived ens33
       valid_lft forever preferred_lft forever
3: ens36: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc fq_codel state UP group default qlen 1000
    altname enp2s4
    altname enx000c29dabfea
    inet 192.168.200.252/24 brd 192.168.200.255 scope global noprefixroute ens36
       valid_lft forever preferred_lft forever
    inet 192.168.200.254/24 scope global secondary proto keepalived ens36
       valid_lft forever preferred_lft forever
root@debian-ha:~#
```

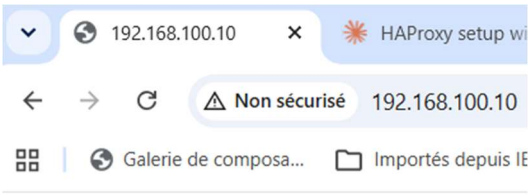
debian_ha porte maintenant ses deux types d'adresses :

Ses IP réelles, 192.168.100.11 et 192.168.200.252, gérées par NetworkManager ;

Les VIP, 192.168.100.10 et 192.168.200.254, marquées secondary proto keepalived. C'est la preuve que Keepalived les a ajoutées et que debian_ha est MASTER.

Test depuis le PC Hôte Windows :

http://192.168.100.10 :

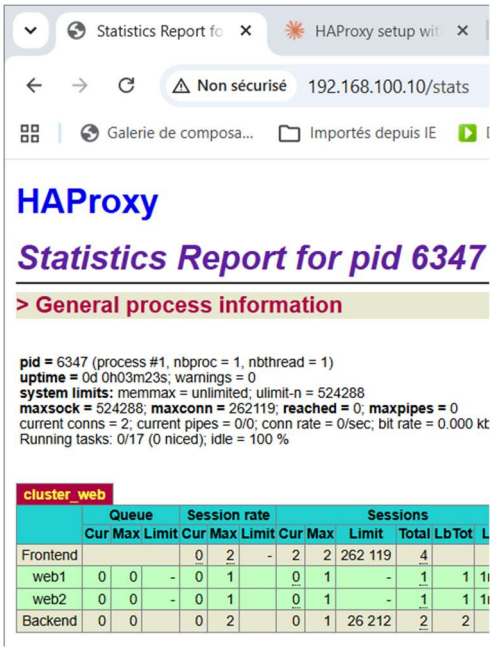


Serveur 1

Ceci est le serveur web 1.

http://192.168.100.10/stats :

admin, mdp : admin



Test depuis debian_web1 :

ping -c 2 8.8.8.8

```
root@debian-web1:~# ping -c 2 8.8.8.8
PING 8.8.8.8 (8.8.8.8) 56(84) bytes of data.
64 bytes from 8.8.8.8: icmp_seq=1 ttl=127 time=6.29 ms
64 bytes from 8.8.8.8: icmp_seq=2 ttl=127 time=8.14 ms

--- 8.8.8.8 ping statistics ---
2 packets transmitted, 2 received, 0% packet loss, time 1002ms
rtt min/avg/max/mdev = 6.288/7.215/8.143/0.927 ms
root@debian-web1:~#
```

Configurer Keepalived sur debian_ha2 (BACKUP) :

cat > /etc/keepalived/keepalived.conf <<'EOF'

global_defs {

  enable_script_security

  script_user root

}

vrrp_script chk_haproxy {

  script "/usr/bin/systemctl is-active --quiet haproxy"

  interval 2

```
    fall 2
    rise 2
}

vrrp_instance VI_1 {
    state BACKUP
    interface ens36
    virtual_router_id 51
    priority 100
    advert_int 1

    authentication {
        auth_type PASS
        auth_pass tpha2026
    }

    virtual_ipaddress {
        192.168.100.10/24 dev ens33
        192.168.200.254/24 dev ens36
    }

    track_script {
        chk_haproxy
    }
}
EOF

systemctl enable --now keepalived
systemctl restart keepalived
```

Vérification sur debian_ha2 :

ip -4 a

journalctl -u keepalived -n 10 --no-pager

```
root@debian-ha2:~# ip -4 a
journalctl -u keepalived -n 10 --no-pager
1: lo: <LOOPBACK,UP,LOWER_UP> mtu 65536 qdisc noqueue state UNKNOWN group default qlen 1000
    inet 127.0.0.1/8 scope host lo
       valid_lft forever preferred_lft forever
2: ens33: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc fq_codel state UP group default qlen 100
0
    altname enp2s1
    altname enx000c292ed641
    inet 192.168.100.12/24 brd 192.168.100.255 scope global noprefixroute ens33
       valid_lft forever preferred_lft forever
3: ens36: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc fq_codel state UP group default qlen 100
0
    altname enp2s4
    altname enx000c292ed64b
    inet 192.168.200.253/24 brd 192.168.200.255 scope global noprefixroute ens36
       valid_lft forever preferred_lft forever
oct. 20 11:52:27 debian-ha2 Keepalived[6571]: Starting Keepalived v2.3.3 (03/30,2025)
oct. 20 11:52:27 debian-ha2 Keepalived[6571]: Running on Linux 6.12.113+deb13-amd64 #1 SMP PREEMPT_
DYNAMIC Debian 6.12.113-1 (2026-10-15) (built for Linux 6.12.21)
oct. 20 11:52:27 debian-ha2 Keepalived[6571]: Command line: '/usr/sbin/keepalived' '--dont-fork'
oct. 20 11:52:27 debian-ha2 Keepalived[6571]: Configuration file /etc/keepalived/keepalived.conf
oct. 20 11:52:27 debian-ha2 Keepalived[6571]: NOTICE: setting config option max_auto_priority shoul
d result in better keepalived performance
oct. 20 11:52:27 debian-ha2 Keepalived[6571]: Starting VRRP child process, pid=6572
oct. 20 11:52:27 debian-ha2 Keepalived[6571]: Startup complete
oct. 20 11:52:27 debian-ha2 systemd[1]: Started keepalived.service - Keepalive Daemon (LVS and VRRP
).
oct. 20 11:52:27 debian-ha2 Keepalived_vrrp[6572]: VRRP_Script(chk_haproxy) succeeded
oct. 20 11:52:27 debian-ha2 Keepalived_vrrp[6572]: (VI_1) Entering BACKUP STATE
root@debian-ha2:~#
```

debian_ha2 est en état BACKUP : il ne porte pas les VIP, qui restent sur debian_ha (MASTER).

Tests de bascule

Test 1 : panne de HAProxy sur le MASTER :

Sur debian_ha :

systemctl stop haproxy

Après quelques secondes :

ip -4 a sur debian_ha : les VIP .10 et .254 ont disparu :

```
root@debian-ha:~# ip -4 a
1: lo: <LOOPBACK,UP,LOWER_UP> mtu 65536 qdisc noqueue state UNKNOWN group default qlen 1000
    inet 127.0.0.1/8 scope host lo
       valid_lft forever preferred_lft forever
2: ens33: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc fq_codel state UP group default qlen 1000
    altname enp2s1
    altname enx000c29dabfe0
    inet 192.168.100.11/24 brd 192.168.100.255 scope global noprefixroute ens33
       valid_lft forever preferred_lft forever
3: ens36: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc fq_codel state UP group default qlen 1000
    altname enp2s4
    altname enx000c29dabfea
    inet 192.168.200.252/24 brd 192.168.200.255 scope global noprefixroute ens36
       valid_lft forever preferred_lft forever
```

ip -4 a sur debian_ha2 : les VIP sont apparues :

```
root@debian-ha2:~# ip -4 a
1: lo: <LOOPBACK,UP,LOWER_UP> mtu 65536 qdisc noqueue state UNKNOWN group default qlen 1000
    inet 127.0.0.1/8 scope host lo
       valid_lft forever preferred_lft forever
2: ens33: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc fq_codel state UP group default qlen 1000
    altname enp2s1
    altname enx000c292ed641
    inet 192.168.100.12/24 brd 192.168.100.255 scope global noprefixroute ens33
       valid_lft forever preferred_lft forever
    inet 192.168.100.10/24 scope global secondary proto keepalived ens33
       valid_lft forever preferred_lft forever
3: ens36: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc fq_codel state UP group default qlen 1000
    altname enp2s4
    altname enx000c292ed64b
    inet 192.168.200.253/24 brd 192.168.200.255 scope global noprefixroute ens36
       valid_lft forever preferred_lft forever
    inet 192.168.200.254/24 scope global secondary proto keepalived ens36
       valid_lft forever preferred_lft forever
```

journalctl -u keepalived -n 5 --no-pager sur debian_ha2 doit afficher Entering MASTER STATE :

```
root@debian-ha2:~# journalctl -u keepalived -n 5 --no-pager
oct. 20 11:56:59 debian-ha2 Keepalived_vrrp[6572]: Netlink reports ens36 down
oct. 20 11:58:12 debian-ha2 Keepalived_vrrp[6572]: Netlink reports ens33 up
oct. 20 11:58:12 debian-ha2 Keepalived_vrrp[6572]: Netlink reports ens36 up
oct. 20 11:58:12 debian-ha2 Keepalived_vrrp[6572]: (VI_1) Entering BACKUP STATE
oct. 20 12:24:01 debian-ha2 Keepalived_vrrp[6572]: (VI_1) Entering MASTER STATE
```

Sur le PC Hôte Windows : http://192.168.100.10/stats affiche « on debian-ha2 » dans le titre, grâce à stats show-node.

HAProxy

Statistics Report for pid 3819 on debian-ha2

> General process information

pid = 3819 (process #1, nbproc = 1, nbthread = 1)
uptime = 0d 1h11m33s; warnings = 2
system limits: memmax = unlimited; ulimit-n = 524288
maxsock = 524288; **maxconn =** 262119; **reached =** 0; **maxpipes =** 0
current conns = 2; current pipes = 0/0; conn rate = 1/sec; bit rate = 102.782 kbps
Running tasks: 0/17 (0 niced); idle = 100 %

cluster_web

	Queue			Session rate			Sessions					
	Cur	Max	Limit	Cur	Max	Limit	Cur	Max	Limit	Total	LbTot	Last
Frontend				1	3	-	2	3	262 119	9		
web1	0	0	-	0	0		0	0	-	0	0	?
web2	0	0	-	0	0		0	0	-	0	0	?
Backend	0	0		0	0		0	0	26 212	0	0	0s

Le serveur répond maintenant sur debian-ha2.

Remettre HAProxy en marche sur debian_ha :

systemctl start haproxy

HAProxy

Statistics Report for pid 8366 on debian-ha

> General process information

pid = 8366 (process #1, nbproc = 1, nbthread = 1)
uptime = 0d 0h00m05s; warnings = 0
system limits: memmax = unlimited; ulimit-n = 524288
maxsock = 524288; **maxconn =** 262119; **reached =** 0; **maxpipes =** 0
current conns = 1; current pipes = 0/0; conn rate = 1/sec; bit rate = 0.000 kbps
Running tasks: 0/16 (0 niced); idle = 100 %

cluster_web

	Queue			Session rate			Sessions				
	Cur	Max	Limit	Cur	Max	Limit	Cur	Max	Limit	Total	Lb
Frontend				1	1	-	1	1	262 119	1	
web1	0	0	-	0	0		0	0	-	0	
web2	0	0	-	0	0		0	0	-	0	
Backend	0	0		0	0		0	0	26 212	0	

Test 2 : On va éteindre complètement le serveur debian_ha :

Préparation du test :

Sur votre PC Windows, dans une invite de commandes :

ping -t 192.168.100.10 (ping continu)

Sur debian_web1 :

ping 8.8.8.8 (ping continu)

Sur debian_ha2 : (-f affiche le journal en direct, pour voir le passage en MASTER au moment où il se produit.)

journalctl -u keepalived -f

La panne

Dans VMware, éteindre brutalement debian_ha : VM → Power → Power Off, et non un arrêt propre. Cela simule une vraie panne, comme une coupure de courant.

Résultat :

Ping Windows : 1 réponse perdue (« Délai d'attente dépassé »), puis les réponses reprennent.

```
Réponse de 192.168.100.10 : octets=32 temps<1ms TTL=64
Réponse de 192.168.100.10 : octets=32 temps<1ms TTL=64
Réponse de 192.168.100.10 : octets=32 temps<1ms TTL=64
Délai d'attente de la demande dépassé.
Réponse de 192.168.100.10 : octets=32 temps<1ms TTL=64
Réponse de 192.168.100.10 : octets=32 temps<1ms TTL=64
Réponse de 192.168.100.10 : octets=32 temps<1ms TTL=64
```

Ping depuis web1 : Aucun ping perdu

Web1 n'a perdu aucun paquet parce que Keepalived envoie un ARP gratuit au moment de la bascule. Il annonce sur le réseau : « l'adresse 192.168.200.254 correspond maintenant à la carte de debian_ha2 ». web1 met immédiatement à jour sa table ARP, et la bascule s'est produite entre deux pings.

Journal de debian_ha2 : (VI_1) Entering MASTER STATE.

```
oct. 20 13:38:42 debian-ha2 Keepalived_vrrp[6572]: Netlink reports ens33 up
oct. 20 13:38:42 debian-ha2 Keepalived_vrrp[6572]: (VI_1) Entering BACKUP STATE
oct. 20 13:49:33 debian-ha2 Keepalived_vrrp[6572]: (VI_1) Entering MASTER STATE
```

HAProxy répartit la charge et assure la redondance des serveurs web ;

Keepalived (VRRP) assure la redondance de HAProxy et de la passerelle, grâce à deux IP virtuelles ;

La bascule est automatique dans les deux sens : panne de HAProxy seul (test 1) et panne de la machine entière (test 2).

Équilibrage des charges

On va demander au serveur debian_web1 de prendre 30% du traffic et debian_web2 70% du traffic.

Modification de la configuration :

Sur debian_ha et debian_ha2, la configuration doit rester identique sur les deux proxys :

Il faut faire la même modification sur debian_ha2, car chaque proxy lit uniquement son propre fichier haproxy.cfg. Ils ne se partagent pas leur configuration : Keepalived ne transfère que les IP virtuelles.

nano /etc/haproxy/haproxy.cfg

Avant :



Après :

```
listen cluster_web
    bind 192.168.100.10:80
    mode http
    balance roundrobin
    option forwardfor

    server web1 192.168.200.11:80 check weight 30
    server web2 192.168.200.12:80 check weight 70
```

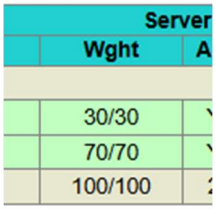
Puis recharger le service :

haproxy -c -f /etc/haproxy/haproxy.cfg

systemctl reload haproxy

Vérification

Sur http://192.168.100.10/stats, la colonne Wght affiche 30/30 pour web1 et 70/70 pour web2. L'équilibre des charges est bien respecté.



Server	Wght	A
	30/30	
	70/70	
	100/100	

Comptage sur 100 requêtes, depuis debian_ha ou debian_ha2 :

for i in $(seq 1 100); do curl -s http://192.168.100.10 | grep -o "Serveur [12]"; done | sort | uniq -c

```
root@debian-ha2:~# for i in $(seq 1 100); do curl -s http://192.168.100.10 | gr
p -o "Serveur [12]"; done | sort | uniq -c
     30 Serveur 1
     70 Serveur 2
root@debian-ha2:~#
```

Protection contre les attaques DDOS :

HAProxy peut servir de bouclier devant les serveurs web afin de limiter les effets des attaques par déni de service. Lors d'une attaque DDoS (Distributed Denial of Service), un très grand nombre de requêtes est envoyé simultanément depuis de nombreuses machines vers un serveur web, qui finit par saturer et ne plus répondre.

Face à une véritable attaque DDoS de grande ampleur, HAProxy n'est pas suffisant : si la connexion Internet elle-même est saturée, il faut faire appel à un service spécialisé qui filtre le trafic en amont, comme Cloudflare ou OVH Anti-DDoS.

HAProxy permet néanmoins de :

Limiter le nombre de requêtes par adresse IP sur une période donnée (rate limiting) : au-delà de 20 requêtes en 10 secondes, le client reçoit une erreur 429 Too Many Requests ;

Se protéger des attaques lentes de type Slowloris, en coupant les connexions qui n'envoient pas leur requête complète dans un délai de 5 secondes ;

Éviter la surcharge des serveurs web, en limitant à 100 le nombre de connexions simultanées envoyées à chacun d'eux.

Nous allons créer un tableau où HAProxy note, pour chaque adresse IP, le nombre de requêtes envoyées sur une période. Au-delà d'un seuil, il répond 429 Too Many Requests au lieu de transmettre la requête.

Sur debian_ha et debian_ha2 :

nano /etc/haproxy/haproxy.cfg

Juste après la ligne option forwardfor, ajouter ces 5 lignes, avec 4 espaces au début comme les autres :

```
    # --- Protection anti-flood ---
    timeout http-request 5s
    stick-table type ip size 100k expire 30s store http_req_rate(10s)
    http-request track-sc0 src
    http-request deny deny_status 429 if { sc_http_req_rate(0) gt 20 }
```

Et ajoutez maxconn 100 à la fin des deux lignes server

Avant :

```
listen cluster_web
    bind 192.168.100.10:80
    mode http
    balance roundrobin
    option forwardfor

    server web1 192.168.200.11:80 check weight 30
    server web2 192.168.200.12:80 check weight 70
```

Après :

```
listen cluster_web
    bind 192.168.100.10:80
    mode http
    balance roundrobin
    option forwardfor
    # --- Protection anti-flood ---
    timeout http-request 5s
    stick-table type ip size 100k expire 30s store http_req_rate(10s)
    http-request track-sc0 src
    http-request deny deny_status 429 if { sc_http_req_rate(0) gt 20 }

    server web1 192.168.200.11:80 check maxconn 100
    server web2 192.168.200.12:80 check maxconn 100

    stats enable
    stats hide-version
    stats refresh 30s
    stats show-node
    stats auth admin:admin
    stats uri /stats
```

On a retiré le ratio 30% et 70%

Recharger :

haproxy -c -f /etc/haproxy/haproxy.cfg

systemctl reload haproxy

Test de la protection :

for i in $(seq 1 30); do curl -s -o /dev/null -w "%{http_code}\n" http://192.168.100.10; done | sort | uniq -c

```
root@debian-ha2:~# for i in $(seq 1 30); do curl -s -o /dev/null -w "%{http_cod
}\n" http://192.168.100.10; done | sort | uniq -c
     20 200
     10 429
```

Les 20 premières requêtes sont passées, avec le code 200 (OK). Les 10 suivantes ont été bloquées par HAProxy, avec le code 429 (Too Many Requests), sans jamais atteindre les serveurs web.

Visualisation du tableau suivi en mémoire :

Sur debian_ha :

apt install -y socat

echo "show table cluster_web" | socat stdio /run/haproxy/admin.sock

Puis relancer le test depuis debian_ha2 :

for i in $(seq 1 30); do curl -s -o /dev/null -w "%{http_code}\n" http://192.168.100.10; done | sort | uniq -c

sur debian_ha :

```
root@debian-ha:~# echo "show table cluster_web" | socat stdio /run/haproxy/admin.sock
# table: cluster_web, type: ip, size:102400, used:2
0x7f5aa8f81450: key=192.168.100.12 use=0 exp=19057 shard=0 http_req_rate(10000)=26
0x7f5aa8f81090: key=192.168.100.1 use=0 exp=17234 shard=0 http_req_rate(10000)=1

root@debian-ha:~# echo "show table cluster_web" | socat stdio /run/haproxy/admin.sock
# table: cluster_web, type: ip, size:102400, used:1
0x7f5aa8f81090: key=192.168.100.1 use=0 exp=16660 shard=0 http_req_rate(10000)=1

root@debian-ha:~#
```

http_req_rate est au-dessus de 20, ici 26, donc la connexion est bloquée.